COS4091 Senior Project Instructions

Version 01

AUBG Computer Science Department

1 Overview

The COS Senior Project is an academic course, just like any other academic course at AUBG. As such, it is subject to the same AUBG rules and regulations governing academic courses, as set out in the AUBG Catalog. This document details the extra regulations for the COS Senior Project as dictated by the COS department. Failure to observe the Catalog's rules and regulations for courses and the regulations as set out here will incur penalties: either a grade reduction or being dropped from the Senior Project.

COS 4091 serves as a capstone project according to ACM/IEEE curriculum recommendations:

> "Capstone and large projects in computing education are used as a vehicle for giving students as close to a 'real-world' experience in software development as possible within the constraints of a computing degree program."

and as a diploma thesis, substituting the State Exam according to Bulgarian law of higher education, the project must demonstrate sufficient expertise in applying competences obtained during the AUBG Computer Science major: all core courses and elective course corresponding to chosen concentration.

In its role as a capstone project, the project MUST demonstrate that the student aiming to graduate is competent in the core content of the AUBG COS curriculum.

Senior projects require development of two components: Computer Application and Project Documentation:

1. Computer Application must demonstrate student's ability to solve a problem by developing a computer application. Specifics guidelines for the application are given as attachments.
2. Project Documentation serves as diploma thesis. Guidelines for format and structure are given as an attachment.

Projects will be evaluated by the Evaluation team of COS full-time faculty members and external experts. The grade will be assigned according to the project's complexity (see the guidelines), completeness of solution, meeting requirements, and the performance during the entire development process.

The following sections and attachments are indivisible parts of these regulations:

- **Section 2:** General project guidelines for the development of the Computer Science Senior Project (Computer Application).
- **Section 3:** Detailed requirements for the project and the course.
- **Section 4:** Guidelines for the structure, format, and content of Project Documentation (Thesis).
- **Section 5:** The process of developing Senior Project.
- **Section 6:** A list of currently disallowed project topics.
- **Third to Last Page:** Declaration of understanding.
- **Last Two Pages:** Templates for the diploma thesis cover and second pages

2 General Project Guidelines

The Senior Project requires the development of a significant software package which must be principally authored by the student. Generally, the project will be taken as an assessment of the student's level of fluency with the problems and solutions of the field of computer science.

2.1 Assessment

This assessment will look for:

1. a nuanced and deep understanding of algorithms and data structures.
2. the ability to compose larger, complex, and correct programs using a specific engineering strategy.
3. the ability to acquire new and relevant knowledge and skills related to the project's topic.
4. the ability to speak and write fluently about the project and all associated work.
5. the evidence of an entire semester's worth of work. (NOTE: Not included in the rubric.)

The previous list should be taken **very** seriously. Your work will be evaluated on how well it attains to these qualities. A rubric (Table 1) is provided below to help you judge your project as you work on it through the semester.

The rubric is a *general* guideline. We understand that every project has different requirements, expectations and critical points. These will be taken into consideration. It is your task throughout the semester to work with your advisor and convince him or her in your weekly meetings, reports and work that you are meeting these requirements.

Please repeatedly refer back to this guideline and rubric.

2.2 Project Topic

Your senior project topic is yours to decide. The topic should belong to one or more specific areas of computer science (e.g., networking, cybersecurity, artificial intelligence, programming languages, compilers, robotics, embedded systems, IoT, etc.). The previous list is not exhaustive and you are

Score	Low Effort	Medium Effort	High Effort
1. Use of data structures and algorithms (DS/A)	Basic data structures and algorithms typically found in sophomore courses. DS/A that are artificially added to the project to meet requirements that do not meaningfully add to the solution. Little or no thought given to time or space complexity. No bench-marking or profiling.	More advanced DS/A not typically found in CS coursework. All algorithms are meaningfully tied to the project. DS/A were chosen with complexity considerations given to critical operations. Some benchmarking and profiling.	Highly complex or composite DS/A designed specifically to optimize the solution. High level of complexity analysis (both theoretical and applied). Multiple solutions are given and discussed. Knowledge of historical solutions are presented and analyzed (if applicable).
2. Engineering and testing strategy	Code exhibits very little organization. Code is messy and hard to read. No explicit design strategy was used. Code documentation (e.g., comments) is sparse or non-informative. Minimal or poor testing of core systems (use case testing, unit testing, load testing, etc.) Bare bones README.	Code is organized, neat and readable. Code is documented with header comments and informative descriptions to promote readability and maintainability. Software design choices are evident in the code. The code base has been tested broadly. Software comes with explicit instructions for compilation and use with a list of dependencies.	Code exhibits a professional level of structure and documentation. The repository's README is complete with detailed instructions and examples and/or results. Testing influenced the software creation process (e.g., test-driven design). Very high provable confidence that code is correct and, where not, there is documentation of known bugs or limitations.
3. Acquire new knowledge and skills	Topic and solution are taken directly from a course taught at AUBG. The project did not require the student to learn anything new (language, additional domain, algorithm, data structure, etc.)	Topic and solution are a mix of known and new material. New material represents the minority of the project. Or new material is itself basic (HTML, CSS, etc.). Or new material is superfluous to the project.	Topic and solution are advanced beyond that found in undergraduate classes. Project required substantial research and skill gain. Project requires substantial interdisciplinary knowledge not previously known.
4. Ability to write and speak about the project	Thesis minimally describes the topic and solution. Thesis contains filler content or excessive repetition of information. Thesis is poorly formatted. The student struggles to speak and answer questions with confidence about the topic, the solution and the choices made throughout the work.	Thesis is well-formatted, concise and captures all aspects of the topic and solution. The thesis itself is sufficient to understand the student's work. The student has a solid grasp of the core of their topic and their work.	The thesis exhibits strong writing skills and good style. The thesis places the work in context with other solutions. The student anticipates weak areas or gaps in the current solution. The student has mastered the material surrounding the topic of their project and is able to speak fluently about it.

Table 1: COS4091 Senior Project Rubric

encouraged to do a bit of reading and research before selecting your topic. We recommend that your topic have the following general properties:

- It is outside (i.e., more advanced than) the work and topics you explored in your computer science courses.
- It can be accomplished in a semester. This means it is neither too long nor too short.
- You are interested in exploring this topic. This is important; if the topic bores you from the beginning, you will find it hard to produce work that *you* are satisfied with (much less us).

Modularity in a topic is recommended. Consider the various components and subproblems you will have to solve. A good project can easily be added to or subtracted from.

Remember, this is a **computer science** project. What does this mean? It means we, the jury, are interested in work concerning fundamental computer science problems. We are not interested in fancy GUIs or web front-ends or e-commerce portals or beautiful visualizations. These are not requirements and do not count for much in the final grade. We advise you to use your time wisely and focus on the computer science aspects. Spending weeks skinning a project in a web app while neglecting the complexity of your solution is not advised.

3 Detailed Requirements

This section contains a detailed list of requirements for the major aspects of the course. Study this list carefully. Failure to satisfy the requirements of these subsections can result in points lost or a failure for the entire course.

3.1 Weekly Meetings, Reports and Code Repository

Students are required to schedule and meet with their advisor every week, starting in the second week. Students are required to file weekly reports on Canvas. Students are required to push code (minimally) weekly to a repository shared with your advisor.

Failure to meet, submit reports or push code without an excuse can result in failure.

Why is this important? Remember point 5 from the Overview section?

A large part of the evaluation of your work, code, and ability comes from these meetings. We need to see how you identify and solve problems. We need to examine your code base as it develops. We are not just looking for a final product; to adequately evaluate your work we need to see how you grow it incrementally by decomposing the grand plan and solving its necessary subproblems.

3.1.1 Weekly Reports

Weekly reports must be filed by Sunday night. They should detail what you worked on in the previous week. They should also describe what you plan to do next week. These reports will form the basis of what we talk about in our weekly meetings. The work you describe should be visible in the code in your repository. Writing that you accomplished something in code without pushing it to the repository is not acceptable.

The reports should be a **half a page to a page in length**. There is no required format for these reports but they should be intelligible. Unfiltered, personal notes or awful writing is not acceptable. Anything can be included: text, lists, figures, code, diagrams, tables, etc. Use these to create talking points or ask questions. If there is something you want to discuss with your advisor, we recommend you mention it prominently in your report. This way we can prepare for the conversation.

3.1.2 Meetings

Again, you are required to meet with your advisor every week. Even if you have not contributed substantially to the project in a given week, you are still required to meet. Your advisor will provide a mechanism or sign-up sheet through which you can choose (or are assigned) days and times.

If you need to miss a meeting, treat it like you would a normal class. Email your advisor prior to the meeting with the reason for your absence.

In the meeting, be prepared to discuss what you have done. Be prepared to explain your code. Be prepared to answer questions about the next step you intend to take. Bring questions or problems for which you want advice. NOTE: we are not here to solve your problems or debug your code. We are generally happy to help when we can but the responsibility is ultimately yours.

We may make recommendations or give suggestions. If we feel things are off-track, we may make demands. We will be clear when this is the case.

3.1.3 Code Repository

By the second weekly report, you must include in every report a link to your code repository. It must be accessible to your advisor.

You must push your current code base before meeting with your advisor each week (ideally, the day of). Yes, we understand that committing and pushing non-working code is not optimal, but the requirements of this project are different from a typical software engineering project. We need to see code as it develops: the additions, subtractions and squashing of bugs.

WARNING: Committing perfect, professional and complete algorithms or data structures all at once is very suspicious in the age of generative AI. We advise that you get in the habit of committing code a few times a day for each day you work on the project. The commits and pushes provide evidence that you did it.

Including code in your project that you yourself have not written but claim as yours is not acceptable and is grounds for failure. Certain tools auto-generate code (e.g., GUI frameworks or database migrations). If such code is used in your project, a) it needs to be documented and b) it does not count toward either the lines of code or the complexity of your project.

3.2 Project

This subsection contains specific requirements for the programmed component of the course. Please discuss these with your advisor at the beginning of the semester. We need to approve not only what you are doing but also how you are doing it.

1. The software should demonstrate the entire software development life-cycle. This includes requirements, architectural design, implementation, analysis, testing, and debugging.
2. Software should be designed using a clear and logical structure consistent with the language used. For example, an object-oriented language should demonstrate the use of at least 3 classes interrelated either through inheritance or composition in a way consistent with the tenets of OOP. If a project contains multiple independent programs, it is possible for each program to use a different design.
3. The programming language(s) used should make sense for the requirements and design of the project. Please discuss your intended language(s) with your advisor and have good reasons for using it. They can veto certain languages if they feel the reason is to trivialize important aspects of your work. For example, we generally discourage the use of Python for many types of projects due to the availability of off-the-shelf libraries that simplify the solution.
4. The core of your project must be programmed completely by you. You must document the use of all libraries and tools and discuss them with your advisor to insure they do not trivialize your project.
5. Functional and non-functional requirements must be given and evidence provided that the implementation satisfies them.
6. The core of your project must have (at a minimum) 500 lines of code. Only certain parts of the code base count toward this total. Each project is different so it is important that you discuss with your advisor which aspects do and do not count toward this total. There are certain project components which, in the majority of cases, will not be counted. HTML, CSS, XML, JSON, configuration files, auto-generated code, and GUI-related code are included in this.

3.3 Midterm Report and Presentation

Around the 8th week of the semester (mid-term) you will be required to upload a minimum of 20 pages of your diploma thesis. These 20 pages can come from any section(s) of the report. It should follow the formatting guidelines above.

The text should be relatively finished. This means, we would like you to focus on producing one or more finished sections by the mid-term. For example, you should have, by this time, a good idea of the problem and your approach. Therefore, you could write a near finished introduction. Or, perhaps you have a strong architectural design. Write that section. Or perhaps the requirements.

What we do not want to see is a full mini-version of your thesis covering all sections. This is a waste of your time as you will have to rewrite most of it to expand it out to its full size. Instead concentrate on a few sections. Push them as close to finished as you can such that, when the end of the semester arrives, these sections will require minimal work.

Also, you will need to upload a 5 minute recorded presentation of yourself with slides discussing your project and the progress made to-date. The presentation does not need to be highly polished. Focus on giving us a clear description of your project. The goal of this presentation is to give you practice talking about your work.

3.4 Final Submission and Presentation

At the end of the semester, during the final week of classes, you will present your project and work to the jury and the public (typically other INF and COS students). The presentation consists of two parts: a typical talk (with slides, for example) covering an overview of your project and a demo that showcases the important aspects of the program. Together these two should take no more than 15 minutes (we recommend about 10 minutes for the talk and 5 for the demo). After the presentation, the jury and the public have approximately 5 minutes for questions. Failure to present or demo your program is grounds for failure of the course.

4 Guidelines for Preparing Project's Documentation (Diploma Thesis)

Along with your completed project, you must deliver a report named, *Diploma Thesis*. This thesis is a comprehensive report on your work written in a professional, technical style.

The thesis must be 60 to 70 pages in length. It can have no more than 20 figures (diagrams, tables, screenshots, images, charts, plots, etc.). Code and pseudo-code should be limited to less than 10 lines.

4.1 Formatting

The formatting requirements of the diploma thesis are:

- Submission file type: MS Word or PDF.
- Paper size: A4
- Margins: 1 inch (or 2.54 cm).
- Font: Arial
- Font size: 12pt
- Line Spacing: 1.5
- Section headings: 14 pt font, bold and underlined.
- Alignment: Fully justified (or both left and right alignment).
- One blank line between paragraphs.
- One blank line above and below section headings.
- Every figure (diagram, table, etc.) must have a caption.
- Diagrams and tables should have a separate numbering sequence.
- No page footer or header.
- Pages should be numbered.

In addition to the requirements above, please see the thesis format provided below. It will give you the layout and general format of the thesis along with suggested page counts for each section.

The following list contains general do's and don'ts for the thesis.

1. If you include code or pseudo-code in your thesis, have a very good reason. If you have created a new algorithm or a variant of an old one, giving the pseudo-code makes sense. But adding code as a way of writing about what your project is doing (rather than writing about it) is frowned upon.
2. Text and figures should flow *nicely*. This means there should not be excessive whitespace padding a figure. If a figure is less than a page in size, that page should also have text on it.
3. Figures should fit on a single page.
4. Figures should be included near where they are first referenced by the text. To this point, figures should be referenced by the text. You should not have figures in your thesis that the text never discusses.
5. Figures should only be as large as the information they are conveying. For example, a simple line plot showing the growth rate of an algorithm with respect to input does not need to take up a whole page.
6. Like code, have a reason for your figures. Make sure they have descriptive captions that explain what the reader should pay attention to. Don't assume the reader knows why a figure is present or the meaning of its content.
7. DO NOT fill your entire report with bullet points. Use them where appropriate.
8. Cite your references so we know where they contribute to your text.

4.2 Senior Project Report Structure

The following lists and describes the required sections of the report and their expected sizes in terms of pages. Additionally, some suggestions concerning what you may write about in each section are given. It is important that you address the title of each section in your writings.

The report must be written in good, grammatical English without spelling mistakes. (The page count that is given for each section is a guideline.) Note that no listing of source code is required in the report.

Title and Declaration Pages

These can be found on Canvas and are included at the end of this document for reference.

0. Table of Contents

The table of contents should list the following sections with their starting page number.

1. Introduction (approx. 2 pages)

A description of the application: an overview of the project. An overview of the technologies used. What the project is about and why you chose it. What interests you about this project.

2. Specification of the Software Requirements and their Analysis (approx. 14 pages)

A description of your application's software requirements, and their analysis.

- A list of the functional and (2-3) non-functional requirements, i.e. software features that are implemented, plus any constraints, written as text sentences.
- Analysis of each functional requirements either via detailed pseudocode descriptions or via detailed UML use cases and activity diagrams.

3. Design of the Software Solution (approx. 20 pages)

A description of your application's software design, including the software architecture

- A description of any special algorithm(s) used.
- A description of the user interface.
- A description of your application's software architecture, i.e. its breakdown into software components, with a description of the services offered by each component, i.e. what each component does and how it interacts with other components. Suitable architectures are layers or MVC. Illustrate which functions classes "live" in which component in accordance with the Separation of Concerns.
- UML component diagrams for the software components of your software architecture
- E-R diagrams for any database tables
- UML deployment diagram for the software components of your application
- Description of the security features and reliability considerations taken into account by your application. E.g. any encryption used for passwords and anti-hacking precautions implemented in your application.

4. Implementation (approx. 12 pages)

Description of the technologies and approach used to implement your software solution.

- The computing platform, programming language(s), external frameworks/libraries, etc. used. Include a justification for use of these.
- Details of installation requirements.
- Code fragments for any particular interesting design/implementation features.
- Libraries and/or frameworks used, if any.

5. Testing (approx. 12 pages)

Write about what testing you have done to convince yourself that the software works as it is supposed to, and does not fail if wrong or no data is passed to it. Acceptance tests are important here. Give some examples of acceptance testing for your main requirements, at least. Generally speaking, for any requirement there would be a number of tests, each designed to show that the requirement is implemented as required. If you have done any other form of testing, e.g. unit tests, give examples of these as well.

6. Results and Conclusion (approx. 4 pages)

What you achieved with your project. How does the final result compare with what you initially hoped to do? What works and what does not? What problems were encountered and how were they resolved, if at all. Screenshots of the application running. Missing features for future development, etc. What you got out of the project; what you have learnt.

7. References (approx. 1 page)

Books, articles, URLs of articles, etc. that you found useful for your project. There should be at least 10 references.

TOTAL PAGES: approximately 65

MIN/MAX PAGES: 60 to 70 pages

NOTE: You can have **at most 20 figures**. This includes diagrams, tables, screenshots, images, pseudo-code, code fragments and other non-textual content. Code fragments should be no more than **approx. 7 lines of code**. All non-textual content should be sized appropriately to the content and such that it is readable. A figure can occupy at most one page. Figures beyond the limit of 20 must be placed in an appendix and do not count toward the 65-page requirement. **Figures and tables must be numbered sequentially and have captions.**

5 Senior Project Development Process

This section gives a chronological overview of the complete process to help you understand the trajectory and requirements of the entire course. Some of the material is repeated from above.

5.1 Registration for the Course

Students must sign for the COS 4091 on Empower before add/drop week.

Students must submit to the instructor assigned for the course a short description of the idea for the project. The student must provide the name of the supervisor (among the full-time COS faculties) in case the student has already reached an agreement with the member of the department.

5.2 Selection of supervisor and the first two weeks

The COS department will consider candidates for developing senior projects and will assign a supervisor to students according to availability.

The supervisor informs the students and arranges: signing the individual agreement about the projects, signing declaration of understanding; setting the times for weekly meetings. During the second week, an information session will be held.

5.3 Weekly Meetings

Students are required to attend at least one regular meeting every week with their project supervisor. The weekly meetings are not optional – they are compulsory. Missing more than three meetings without the prior permission of the supervisor may lead to the student being dropped from the course.

After each weekly meeting (by the following Sunday, the student must upload to the Canvas COS course website a summary of the weekly progress – between half- and one page. There will be penalties for not uploading these weekly summaries.

5.4 Mid-term Progress Report and Presentation

You need to prepare and deliver a presentation, via a recording of yourself, (approx. 10 PPT slides, 5 min maximum) of your project progress and how far you have come with the development. Additionally, you must also upload on Canvas at least 20 pages of your final report (Bulgarian Diploma Thesis) by this deadline. These 20 pages can be from different parts of your project.

5.5 Final Submission, Deliverables, Defense, and Evaluation

The deadline for the upload to Canvas of the Diploma Thesis report is the 12th week (the exact deadline will be announced). The electronic report must be accompanied by a zip file, which is also to be uploaded to Canvas. The contents of the zip file are to be a copy of all the project's source code files, executable files, installation files, etc.

You must deliver a short 10-min presentation of your project during the final teaching week. You must also deliver a demo of the project. You will be sent a sign-up sheet for you to choose a day for your presentation. This will be organized on a first-come/first-served basis.

After submission report work on COS Senior Project must be frozen! There will be a penalty for late or no delivery of the reports and/or the zip files. Non-delivery of the final presentation or the demo will result in an "F" grade for the Senior Project.

6 Currently Disallowed Projects

The following is a list of disallowed project topics or formats. If you feel your project resembles something on this list, bring it to the attention of your advisor. These are here for a reason, but not all the same reason. Some represent perennial low-effort projects. Some have been done too often in the recent past.

- Websites: Web views can be used for displaying results as a kind of output format, but a simple website is not appropriate for a COS project.
- Chatbots: Unless you have the resource to train your own LLM, chatbots are a high-level problem requiring extensive low-level NLP problem-solving that are too large for the scope of the senior project.

Declaration for understanding

I, _____, ID#_____,
(student's name and ID)

have read and understand
the requirements, conditions and consequences of the above
regulations in developing the COS Senior Project
in the Spring semester of 2025-26 academic year.

_____	_____
(Student Signature)	(Advisor Signature)
_____	_____
(Student Name)	(Advisor Name)
_____	_____
(Date)	(Date)

Cover page template for Diploma Thesis

Bulgarian Diploma Thesis

Title

Your Name, ID#

Student: _____ **, Date:** _____

signature

Supervisor: _____ **, Date:** _____

signature

Department of Computer Science, AUBG

Blagoevgrad, 2025

Declaration template for Senior Project and Diploma Thesis

Title:

Author:

Abstract: half-page description

Declaration of authorship:

“The Senior Project/Bulgarian Diploma Thesis presented here is the work of the author solely, without any external help, under the supervision of ….. All sources, used in development, are cited in the text and in the Reference section.”

Author: _____